

Steps to create (or recreate) a Standby Cluster using a Snapshot from a Production cluster as Master cluster (instead of pg_basebackup)

Summary

From time to time we have to create Standby Clusters from a source/existing Gitlab.com Patroni database, usually from GPRD or GSTG.

Standby Clusters are physically replicated clusters, that stream or recover WALs from the source Patroni/PostgreSQL database, but have an independent Patroni configuration and management, therefore can be promoted if required.

This runbook describes the whole procedure and several takeaways to help you to create a new Patroni Standby Clusters from scratch.

1. Pre-requisites

1. Terraform should be installed and configured;
2. Ansible should be installed and configured into your account into your workstation or a `console` node, you can use the following commands:

```
python3 -m venv ansible
source ansible/bin/activate
python3 -m pip install --upgrade pip
python3 -m pip install ansible
ansible --version
```

3. Download/clone the `ops.gitlab.net/gitlab-com/gl-infra/config-mgmt` project into your workstation or a `console` node;

2. Chef role for the Target cluster

Some `postgresql` settings need to be the SAME as the Source cluster for the physical replication to work, they are:

```
"postgresql": {
  "version":
  "pg_user_homedir":
  "config_directory":
  "data_directory":
  "log_directory":
  "bin_directory":
  ...
}
```

IMPORTANT: the `gitlab_walg.storage_prefix` in the target Chef role **SHOULD NOT BE THE SAME** as the Source cluster, otherwise the

backup of the source cluster can be overwritten.

The Chef role of the standby patroni cluster should have defined the `standby_cluster` settings under `override_attributes.gitlab-patroni.patroni.config.bootstrap.dcs` like the below example. Notice that `host` should point to the endpoint of the Primary/Master node of the source cluster, therefore if there's a failover we don't have to reconfigure the standby cluster.

```
"override_attributes": {
  "gitlab-patroni": {
    "patroni": {
      "config": {
        "bootstrap": {
          "dcs": {
            "standby_cluster": {
              "host": "master.patroni.service.consul",
              "port": 5432
            }
          }
        }
      }
    }
  }
},
```

3. Define the new Standby Cluster in Terraform

Define a disk snapshot from the source cluster in Terraform, for example:

```
data "google_compute_snapshot" "gcp_database_snapshot_gprd_main_2004" {
  filter      = "sourceDisk eq .*/patroni-main-2004-.*"
  most_recent = true
}
```

:warning: IMPORTANT: Use an existing snapshot or create a new one manually – to do the latter properly, follow the procedure defined in `gcs-snapshots.md`. If you create a new GCS snapshot without proper additional actions, Postgres might not be able to reach the consistency point (it won't start).

When defining the target cluster module in Terraform, define the storage size and settings similar to the source, and then define the `data_disk_snapshot` pointing to the source snapshot and a large amount of time for `data_disk_create_timeout`, like for example:

```
module "patroni-main-standby_cluster" {
  source = "ops.gitlab.net/gitlab-com/generic-stor-with-group/google"
  version = "8.1.0"

  data_disk_size = var.data_disk_sizes["patroni-main-2004"]
```

```

data_disk_type          = "pd-ssd"
data_disk_snapshot      = data.google_compute_snapshot.gcp_database_snapshot_gprd_main_20
data_disk_create_timeout = "120m"
...
}

```

4. Steps to Destroy a Standby Cluster if you want to recreate it

IMPORTANT: make sure to review the MRs and commands or perform the execution with a peer

IMPORTANT: You should NEVER perform this operation for existing clusters **PRODUCTION** clusters that are in use by the application, only destroy new clusters you are rebuilding for new projects;

If the cluster exists and is not operational, in sync, or has issues with the source replication, create an MR to destroy the cluster. As a standard practice, always rebase MR before merging it and review MR's Terraform plan job for accuracy.

Once MR is merged, clean out any remaining Chef client/nodes using `knife`, like for example:

```

knife node delete --yes patroni-main-standby_cluster-10{1..5}-db-$env.c.gitlab-$gcp_project
knife client delete --yes patroni-main-standby_cluster-10{1..5}-db-$env.c.gitlab-$gcp_project

```

5. Create the Patroni CI Standby Cluster instances

5.1. Create the cluster with Terraform MR

If you are creating the nodes for the first time, **they should be created by our CI/CO pipeline** when you merge the changes in `main.tf` in the repository.

Otherwise, create a new MR to revert the changes performed by the earlier MR that destroyed it. Again, as a standard practice, always rebase MR before merging it and review MR's Terraform plan job for accuracy. Merge the MR to create cluster nodes.

5.2. Stop patroni and reset WAL directory from old files

Before executing the playbook to create the standby cluster, you have to stop patroni service in all nodes of the new standby cluster.

```
knife ssh "role:<patroni_standby_cluster_role>" "sudo systemctl stop patroni"
```

Then you have to clean out the `pg_wal` directory of all nodes of the new standby cluster, otherwise, there could be old TL history data on this directory that will affect the WAL recovery from the source cluster. You can perform the following:

```
knife ssh "role:<patroni_standby_cluster_role>" "sudo rm -rf /var/opt/gitlab/postgresql/data"
```

Note: you can change `/var/opt/gitlab/postgresql/data14` to any other data directory that is in use, eg. `/var/opt/gitlab/postgresql/data16`, etc.

5.3. Initialize Patroni standby_cluster with Ansible playbook

1st - Download/clone the `gitlab.com/gitlab-com/gl-infra/db-migration` project into your workstation or a `console` node;

```
git clone https://gitlab.com/gitlab-com/gl-infra/db-migration.git
```

2nd - Check that the inventory file for your desired environment exists in `db-migration/pg-replica-rebuild/inventory/` and it's up-to-date with the hosts you're targeting. The inventory file should contain:

- `all.vars.walg_gs_prefix`: this is the GCS bucket and directory of the SOURCE database WAL archive location (the source database is the cluster you referred the `data_disk_snapshot` to create the cluster throughout TF). You can find this value in the source cluster Chef role, it should be the `gitlab_walg.storage_prefix` for that cluster.
- `all.hosts`: a regex that represents the FQDN of the hosts that are going to be part of this cluster, where the first node will be created as Standby Leader.

Example:

```
all:
  vars:
    walg_gs_prefix: 'gs://gitlab-gprd-postgres-backup/pitr-walg-main-v14'
  hosts:
    patroni-main-v14-[101:105]-db-gstg.c.gitlab-staging-1.internal:
```

3rd - Run `ansible -i inventory/<file> all -m ping` to ensure that all hosts in the inventory are reachable;

```
cd db-migration/pg-replica-rebuild
ansible -i inventory/<file> all -m ping
```

4th - Execute the `rebuild-all` Ansible playbook to create the `standby_cluster`, and sync all nodes with the source database;

```
cd db-migration/pg-replica-rebuild
ansible-playbook -i inventory/patroni-main-v14-gstg.yml rebuild-all.yml
```

5.4. Check if the Patroni standby_cluster is healthy and replicating

5.4.1. Check standby_cluster source configuration Execute

```
gitlab-patronictl show-config
```

The output should present the `standby_cluster` block with the `host` property pointing to the proper source cluster master endpoint.

5.4.2. Check Replication status Execute

```
gitlab-patronictl list
```

In the output, the leader node of the new standby cluster should have its **Role** defined as **Standby Leader**, the TL (timeline) should match the TL from the source cluster, and all replicas **State** should be **running**, which mean that they are replicating/streaming from their sources.

6. Build DR Archive and Delayed replicas

Once in a while we may find ourselves creating/rebuilding DR archive and delayed replicas from a source/existing Gitlab.com Patroni database, usually from GPRD or GSTG.

DR archive and delayed replicas are physically replicated clusters, that stream or recover WAL segments archived (by an independent Patroni/PostgreSQL database) in an object storage (in our case, GCP).

This runbook describes the whole procedure and several takeaways to help you to create a new Patroni DR archive and delayed replicas from scratch.

6.1 Create the cluster with Terraform MR

When building the delayed and archive replicas you may be required to create the replica instances from scratch or you might be rebuilding an already existing instance.

If you are building from scratch you need to add a module in **config-mgmt** to provision the new instances example MR. If you are rebuilding an existing instance there are two steps:

1. Create an MR to destroy the existing instance this can be done by commenting out the module that provisions the delayed and archive replicas.
2. Create an MR to rebuild the replicas.

NOTE: Remember to always rebase the MR and review the plan before merging.

6.1.1 Chef role for the replicas Before you create a VM instance you need a chef role for the VM instance. In the chef-repo create a role for the DR replica.

For the archive replica role make sure to include the following **dcs** object (stands for Distributed Configuration Store) under the **gitlab-patroni.patroni.config.bootstrap** structure, it stores the dynamic configurations to be applied on all cluster nodes (official patroni documentation).

```
"dcs": {  
  "standby_cluster": {  
    "restore_command": "/usr/bin/envdir /etc/wal-g.d/env /opt/wal-g/bin/wal-g wal-fetch --tu
```

```

        "create_replica_methods": [
            "wal_g"
        ],
    },
    "postgresql": {
        "wal_g": {
            "no_master": true,
            "command": "bash -c '/usr/bin/envdir /etc/wal-g.d/env /opt/wal-g/bin/wal-g backup-list'
        }
    }
}

```

For the delayed replica use the following `dcs` object:

```

"dcs": {
    "standby_cluster": {
        "restore_command": "/usr/bin/envdir /etc/wal-g.d/env /opt/wal-g/bin/wal-g wal-fetch --tu
        "create_replica_methods": [
            "wal_g"
        ],
        "recovery_min_apply_delay": "8h"
    },
    "postgresql": {
        "wal_g": {
            "no_master": true,
            "command": "bash -c '/usr/bin/envdir /etc/wal-g.d/env /opt/wal-g/bin/wal-g backup-list'
        }
    }
}

```

NOTE: The inclusion of `"recovery_min_apply_delay": "8h"` for the delayed replica.

Include the following `override_attributes` as well to both the delayed and archive replicas.

```

"override_attributes": {
    "gitlab-patroni": {
        "postgresql": {
            "parameters": {
                "archive_command": "/bin/true",
                "archive_mode": "off",
                "shared_buffers": "8GB",
                "max_standby_archive_delay": "120min",
                "max_standby_streaming_delay": "120min",
                "statement_timeout": "15min",
                "idle_in_transaction_session_timeout": "10min",
                "hot_standby_feedback": "off"
            }
        }
    }
}

```

```

    }
  }
},
"omnibus-gitlab": {
  "run_reconfigure": false,
  "gitlab_rb": {
    "gitlab-rails": {
      "db_host": "/var/opt/gitlab/postgresql",
      "db_port": 5432,
      "db_load_balancing": false
    },
    "consul": {
      "enable": false
    },
    "postgresql": {
      "enable": false
    },
    "repmgr": {
      "enable": false
    },
    "repmgrd": {
      "enable": false
    },
    "pgbouncer": {
      "enable": false
    }
  }
}
}

```

The role for the DR replicas are similar to the roles of other replicas but with the above `dc`s and override attributes. Remember to also set unique consul service names (for consul service discovery) as well as unique Prometheus type (for monitoring purposes) (example MR). Since we are also specifying attributes for Omnibus we should also include the `recipe[omnibus-gitlab::default]` recipe in the runlist.

6.1.2A Building the replica instance from scratch

- Create an MR to add a module in config-mgmt in the environment's `main.tf` file, this module provisions the VM instance refer here for more details on what to include in the module.
- Merge the MR and wait for the apply pipeline to complete.
- Once apply has completed (sometimes it may take upto 40 minutes, if you are executing this in a CR remember to account for that) check if the nodes are available on chef using the `knife search "role:<role>"`

command like in the example shown below.

```
knife search "role:gstg-base-db-patroni-ci-archive-v14 OR role:gstg-base-db-patroni-ci-delay
2 items found
```

```
postgres-ci-v14-dr-delayed-01-db-gstg.c.gitlab-staging-1.internal
postgres-ci-v14-dr-delayed-01-db-gstg.c.gitlab-staging-1.internal
```

- Once the nodes appear you can tail the serial output logs as you wait for chef to converge. `gcloud compute --project=<project> instances tail-serial-port-output <vm-instance> --zone=<zone> --port=1`

6.1.2B Rebuilding an already existing replica

You start by destroying the old replica using these steps.

Before you recreate the VM instances you need to delete the OLD VMs from chef client and nodes as shown in the example below. Sometimes the nodes are deleted automatically this is just a precautionary step.

Get the nodes you wish to remove from chef client and node list.

```
$ knife node list | grep "ci-v14-dr-archive\|ci-v14-dr-delayed"
```

```
postgres-ci-v14-dr-archive-01-db-gstg.c.gitlab-staging-1.internal
postgres-ci-v14-dr-delayed-01-db-gstg.c.gitlab-staging-1.internal
```

Delete the nodes from chef client.

```
$ knife client bulk delete "^postgres-ci-v14-dr-[a-z]+-01-db-gstg\.c\.gitlab-staging-1\.intern
The following clients will be deleted:
```

```
postgres-ci-v14-dr-archive-01-db-gstg.c.gitlab-staging-1.internal
postgres-ci-v14-dr-delayed-01-db-gstg.c.gitlab-staging-1.internal
```

Are you sure you want to delete these clients? (Y/N) y

Deleted client postgres-ci-v14-dr-archive-01-db-gstg.c.gitlab-staging-1.internal

Deleted client postgres-ci-v14-dr-delayed-01-db-gstg.c.gitlab-staging-1.internal

Delete the nodes from chef node list.

```
$ knife node bulk delete "^postgres-ci-v14-dr-[a-z]+-01-db-gstg\.c\.gitlab-staging-1\.intern
The following nodes will be deleted:
```

```
postgres-ci-v14-dr-archive-01-db-gstg.c.gitlab-staging-1.internal
postgres-ci-v14-dr-delayed-01-db-gstg.c.gitlab-staging-1.internal
```

Are you sure you want to delete these nodes? (Y/N) y

Deleted node postgres-ci-v14-dr-archive-01-db-gstg.c.gitlab-staging-1.internal

Deleted node postgres-ci-v14-dr-delayed-01-db-gstg.c.gitlab-staging-1.internal

After that follow the steps above to recreate it and wait for chef to converge.

6.2 Initialize the patroni DR replica

Log into each of the VM instances and excute the following steps:

```
consul kv delete -recurse service/$(/usr/local/bin/gitlab-patronictl list -f json | jq -r '
sudo rm -f /var/opt/gitlab/postgresql/data14
sudo systemctl start patroni
sudo chef-client
```

Confirm that the patroni starts as Standby Leader:

Execute `sudo gitlab-patronictl show-config` You should see an output similar to the one below.

```
loop_wait: 10
maximum_lag_on_failover: 1048576
postgresql:
  md5_auth_cidr_addresses:
  - 10.0.0.0/8
  parameters:
    checkpoint_timeout: 10min
    hot_standby: 'on'
    max_connections: 670
    max_locks_per_transaction: 128
    max_replication_slots: 32
    max_wal_senders: 32
    max_wal_size: 16GB
    wal_keep_segments: 8
    wal_keep_size: 8192MB
    wal_level: logical
  use_pg_rewind: true
  use_slots: true
  wal_g:
    command: bash -c '/usr/bin/envdir /etc/wal-g.d/env /opt/wal-g/bin/wal-g backup-list | g
      backup-fetch /var/opt/gitlab/postgresql/data14 {}'
    no_master: true
  retry_timeout: 40
  standby_cluster:
    create_replica_methods:
    - wal_g
    recovery_min_apply_delay: 8h
    restore_command: /usr/bin/envdir /etc/wal-g.d/env /opt/wal-g/bin/wal-g wal-fetch --turbo %
  ttl: 90
```